Sistema de Asistencia y Organizacion de Clases

Trabajo Practico Final - Programacion II (2026)

Documento tecnico generado en base al codigo fuente del repositorio.
Version: Noviembre 2026

---

1. Descripcion General del Sistema

Aplicacion de escritorio en Java para que un profesor pueda organizar sus materias, comisiones y tomar asistencia de alumnos de forma eficiente.
Funciona sin conexion a Internet ni login, almacenando datos en MySQL via JDBC.

Objetivo principal: permitir llegar rapido a la comision y tomar lista
sin pasos innecesarios.

2. Estructura de Paquetes Actual

```
src/
  Main.java
  createTables.sql
  comun/
    enums/     DiaSemana.java  |  EstadoAsistencia.java
    excepcion/ EntidadNoEncontradaException.java
               PersistenciaException.java  |  ValidacionException.java
    modelo/    Entidad.java (abstracta)  |  Persona.java (abstracta)
    persistencia/ ConnectDB.java  |  Repositorio.java (interfaz generica)
    util/      Verificador.java
  features/
    agenda/modelo/  EventoAgenda.java (abstracta)  |  Recordatorio.java
    alumnos/modelo/ Alumno.java
    alumnos/persistencia/ AlumnoRepositorio.java
    ayudantes/modelo/ Ayudante.java
    ayudantes/persistencia/ AyudanteRepositorio.java
    clases/modelo/ Asistencia.java  |  Clase.java  |  Material.java
    establecimientos/modelo/ Establecimiento.java  |  Horario.java
    materias/modelo/ Comision.java
    materias/modelo/materia/model/ Materia.java
    materias/modelo/materia/persistencia/ MateriaRepositorio.java
```

3. Clases del Sistema

3.1 Capa Comun (comun/)

Entidad (abstracta): clase base de todas las entidades persistibles.
- Atributos: id (int)
- Regla: id == 0 significa entidad nueva, aun no guardada.
- Metodos: getId(), setId(), esNueva(), equals(), hashCode()

Persona (abstracta, extends Entidad): factoriza campos de Alumno y Ayudante.
- Atributos: nombre, apellido, dni, email, telefono

Repositorio<T extends Entidad> (interfaz generica): contrato CRUD.
- listar(), buscarPorId(int), guardar(T), actualizar(T), eliminar(int)

ConnectDB: Singleton de conexion a MySQL (localhost:3307/testTPP).
Verificador: Utilidad de validacion de entradas por consola.

Enumeraciones:
  EstadoAsistencia: SIN_REGISTRAR | PRESENTE | AUSENTE | TARDE | JUSTIFICADA
  DiaSemana: Lunes | Martes | Miercoles | Jueves | Viernes | Sabado | Domingo

Excepciones personalizadas:
  PersistenciaException   - error en operaciones de BD
  EntidadNoEncontradaException - entidad buscada no existe
  ValidacionException   - datos invalidos (ej. horario inconsistente)

3.2 Feature Agenda

EventoAgenda (abstracta, extends Entidad): base para Clase y Recordatorio.
  - Atributos: fechaHora (LocalDateTime), titulo, descripcion, comision
Recordatorio (extends EventoAgenda):
  - Atributos: clase (nullable), material (nullable), realizado (boolean)

3.3 Feature Alumnos

Alumno (extends Persona):
  - Atributos: legajo (String), comisiones (List<Comision>)
  - Metodos: agregarComision(), quitarComision()
AlumnoRepositorio (implementa Repositorio<Alumno>):
  - CRUD completo y funcional con JDBC.
  - Nota: DNI y legajo NO se pueden actualizar.

3.4 Feature Ayudantes

Ayudante (extends Persona):
  - Atributos: area, alumnoVinculado (Alumno, nullable), comisiones
  - Metodos: agregarComision(), quitarComision()
AyudanteRepositorio (implementa Repositorio<Ayudante>):
  - CRUD completo y funcional. Usa LEFT JOIN con tabla alumno.
  - Validacion: alumno vinculado debe estar guardado (id > 0).

3.5 Feature Clases

Clase (extends EventoAgenda):
  - Atributos: tema, asistencias (Map<Alumno, Asistencia>), materiales (List<Material>)
  - Hereda: fechaHora, titulo, descripcion, comision
  - agregarMaterial() PENDIENTE (cuerpo vacio en codigo actual)
Asistencia (NO extends Entidad, clave compuesta clase_id+alumno_id en BD):
  - Atributos: alumno, clase, estado (EstadoAsistencia), observacion
Material (extends Entidad):
  - Atributos: nombre, descripcion, llevado (boolean), clase

3.6 Feature Establecimientos

Establecimiento (extends Entidad):
  - Atributos: nombre, direccion, localidad
Horario (extends Entidad):
  - Atributos: dia (DiaSemana), horaInicio, horaFin, establecimiento, comision
  - Validacion en constructor: horaInicio < horaFin y comision != null

3.7 Feature Materias

Materia (extends Entidad):
  - Atributos: codigo (unico), nombre, comisiones (List<Comision>)
Comision (extends Entidad):
  - Atributos: nombre, materia, establecimiento, alumnos, horarios, clases
  - agregarAlumno(), quitarAlumno() implementados
  - agregarHorario(), quitarHorario(), agregarClase(), quitarClase() PENDIENTES
MateriaRepositorio (implementa Repositorio<Materia>):
  - guardar() y buscarPorId() funcionales

- listar() devuelve lista vacia; actualizar() y eliminar() vacios

4. Base de Datos - Tablas SQL (testTPP, MySQL utf8mb4)

Tabla	Descripcion	Claves Foraneas
materia	Materias academicas	--
establecimiento	Lugares fisicos	--
alumno	Alumnos registrados	--
ayudante	Ayudantes	-> alumno (opcional)
comision	Comisiones de una materia	-> materia, establecimiento
comision_alumno	N:M comision-alumno	-> comision, alumno
comision_ayudante	N:M comision-ayudante	-> comision, ayudante
horario	Horarios semanales	-> comision, establecimiento
clase	Encuentros concretos	-> comision
material	Materiales para clase	-> clase
asistencia	Asistencia alumno-clase	-> clase, alumno
recordatorio	Recordatorios con FK opcnls	-> comision, clase, material

5. Estado Actual del Proyecto

Implementado y funcional:

- [OK] Modelo de dominio completo (todas las clases)
- [OK] createTables.sql con todas las tablas
- [OK] AlumnoRepositorio - CRUD completo con JDBC
- [OK] AyudanteRepositorio - CRUD completo con JDBC (LEFT JOIN con alumno)
- [OK] MateriaRepositorio - guardar() y buscarPorId() funcionales
- [OK] ConnectDB - singleton de conexion MySQL
- [OK] Verificador - validacion de entradas por consola
- [OK] 3 excepciones personalizadas
- [OK] Enumeraciones EstadoAsistencia y DiaSemana
- [OK] Jerarquia: Entidad -> Persona -> {Alumno, Ayudante}
- [OK] Jerarquia abstracta: Entidad -> EventoAgenda -> {Clase, Recordatorio}
- [OK] Interfaz generica Repositorio<T extends Entidad>

Incompleto / Pendiente:

- [!] MateriaRepositorio.listar() -> Retorna lista vacia
- [!] MateriaRepositorio.actualizar() -> Cuerpo vacio
- [!] MateriaRepositorio.eliminar() -> Cuerpo vacio
- [!] ComisionRepositorio -> No existe aun
- [!] EstablecimientoRepositorio -> No existe aun
- [!] HorarioRepositorio -> No existe aun
- [!] ClaseRepositorio -> No existe aun
- [!] AsistenciaRepositorio -> No existe aun
- [!] MaterialRepositorio -> No existe aun
- [!] RecordatorioRepositorio -> No existe aun
- [!] Clase.agregarMaterial() -> Metodo vacio (// COMPLETAR)
- [!] Comision.agregarHorario/Clase -> No implementados
- [!] Menu interactivo por consola -> No existe aun
- [!] Typo en MateriaRepositorio.mapear() -> 'cdigo' en lugar de 'codigo'

6. Pasos a Futuro (Plan de Desarrollo)

Fase 1 - Completar repositorios faltantes

- MateriaRepositorio: listar(), actualizar(), eliminar(), corregir typo 'cdigo'
- ComisionRepositorio: CRUD + N:M con comision_alumno y comision_ayudante
- EstablecimientoRepositorio: CRUD simple
- HorarioRepositorio: CRUD, mapear DiaSemana y LocalTime
- ClaseRepositorio: CRUD, mapear LocalDateTime, cargar materiales y asistencias
- AsistenciaRepositorio: operaciones sobre clave compuesta (clase_id, alumno_id)
- MaterialRepositorio: CRUD, asociar a clase

- RecordatorioRepositorio: CRUD, 3 FK opcionales

Fase 2 - Completar metodos del modelo

- Clase.agregarMaterial(Material m)
- Comision.agregarHorario(), quitarHorario(), agregarClase(), quitarClase()
- Materia.quitarComision() - revisar validaciones
- Corregir paquete de Materia (materia/model/ -> materia/modelo/)

Fase 3 - Menu interactivo por consola

- Clase Menu principal con opciones numeradas usando Verificador
- Submenu por entidad: Materias, Comisiones, Alumnos, Ayudantes, Clases, Asistencia
- Flujo: seleccionar materia -> comision -> clase del dia -> tomar asistencia
- CRUD de alumnos, ayudantes, establecimientos desde menu
- Ver recordatorios pendientes al iniciar la aplicacion

Fase 4 - Mejoras y pulido

- Documentar todas las clases con Javadoc
- Agregar validaciones faltantes (campos nulos, formatos)
- Implementar backup/exportacion de datos (opcional)
- Notificaciones de recordatorios vencidos al iniciar (opcional)

7. Diagrama Entidad-Relacion (DER)

```
materia (id PK, codigo UNIQUE, nombre)
  |-- 1:N --> comision (id PK, nombre, materia_id FK, establecimiento_id FK)

comision <-- N:M --> alumno  (via tabla comision_alumno)
comision <-- N:M --> ayudante (via tabla comision_ayudante)

alumno (id PK, nombre, apellido, dni UNIQUE, email, telefono, legajo UNIQUE)
ayudante (id PK, nombre, apellido, dni UNIQUE, area, alumno_vinculado_id FK nullable)

establecimiento (id PK, nombre, direccion, localidad)
  <-- FK desde horario y comision

horario (id PK, dia ENUM, hora_inicio, hora_fin, establecimiento_id FK, comision_id FK)

clase (id PK, fecha_hora, titulo, descripcion, tema, comision_id FK)
  |-- 1:N --> material (id PK, nombre, descripcion, llevado, clase_id FK)
  |-- 1:N --> asistencia (clase_id PK+FK, alumno_id PK+FK, estado ENUM, observacion)

recordatorio (id PK, fecha_hora, titulo, descripcion, realizado,
              comision_id FK nullable, clase_id FK nullable,
              material_id FK nullable)
```

8. Diagrama UML de Clases (simplificado)

```
<<abstract>> Entidad
  - id: int
  + getId(), setId(), esNueva(), equals(), hashCode()
  |
  +--<<abstract>> Persona              +--<<abstract>> EventoAgenda
  |   - nombre, apellido, dni, email   |   - fechaHora, titulo, descripcion, comision
  |   |                                |   |
  |   +-- Alumno                       |   +-- Clase
  |   |     - legajo                   |   |     - tema
  |   |     - comisiones:List          |   |     - asistencias:Map<Alumno,Asistencia>
  |   |                                |   |     - materiales:List<Material>
  |   +-- Ayudante                     |
  |         - area                     |   +-- Recordatorio
  |         - alumnoVinculado          |         - clase (nullable)
  |         - comisiones:List          |         - material (nullable)
  |                                    |         - realizado:boolean
```

```
  +-- Comision                          +-- Material
  |     - nombre, materia               |     - nombre, descripcion, llevado
  |     - establecimiento               |
  |     - alumnos, horarios, clases     +-- Establecimiento
  |                                     |     - nombre, direccion, localidad
  +-- Materia                           +-- Horario
  |     - codigo, nombre                      - dia:DiaSemana
  |     - comisiones:List                     - horaInicio, horaFin
  |                                           - establecimiento, comision

<<interface>> Repositorio<T extends Entidad>
  + listar():List<T>  buscarPorId(int):T
  + guardar(T)  actualizar(T)  eliminar(int)
  |
  +-- AlumnoRepositorio    [COMPLETO]
  +-- AyudanteRepositorio  [COMPLETO]
  +-- MateriaRepositorio   [PARCIAL]
  +-- ComisionRepositorio  [PENDIENTE]
  +-- ... (todos los demas) [PENDIENTE]

<<enum>> EstadoAsistencia: SIN_REGISTRAR | PRESENTE | AUSENTE | TARDE | JUSTIFICADA
<<enum>> DiaSemana: Lunes | Martes | Miercoles | Jueves | Viernes | Sabado | Domingo

Asistencia (NO extends Entidad):
  - alumno:Alumno  clase:Clase  estado:EstadoAsistencia  observacion:String
```

9. Cumplimiento de Requisitos de la Consigna

Requisito	Estado
Diagrama UML	[OK] Disenado (ver seccion 8)
4 pilares POO	[OK] Herencia, Polimorfismo, Abstraccion, Encapsulamiento
Min 5 clases	[OK] Superado (13+ clases)
1 Clase Abstracta	[OK] Entidad, Persona, EventoAgenda
1 Interfaz	[OK] Repositorio<T>
1 Enumeracion	[OK] EstadoAsistencia, DiaSemana
Colecciones	[OK] List<> en Comision, Alumno, Ayudante, Materia
Mapa	[OK] Map<Alumno, Asistencia> en Clase
2+ excepciones personalizadas	[OK] 3 excepciones
Clase generica	[OK] Repositorio<T extends Entidad>
Persistencia JDBC	[!!] Parcial (Alumno/Ayudante OK; faltan otros)
Git/GitHub	[OK] Repositorio en uso
Menu interactivo	[!!] PENDIENTE
Documentacion/Comentarios	[!!] Parcial